

Université de La Manouba
École Supérieure d'Économie Numérique
Année universitaire 2025/2026

RAPPORT DE PROJET BI

Projet : Analyse de retours Produits



Présenté dans le cadre du projet

Élaboré par :

Marhag Mariem
2ème année Licence Business Intelligence – Groupe 2

Smati Safa
2ème année Licence Business Intelligence – Groupe 2

Encadré par :

Mme Mouna Chebah, ESEN

Sommaire — Projet BI :

1. Objectif du Projet

- 1.1 Contexte du projet
- 1.2 Objectif principal
- 1.3 Bénéfices attendus

2. Présentation des Données

- 2.1 Nom du dataset
- 2.2 Description brève du dataset
- 2.3 Problématique métier visée
- 2.4 Relations entre les tables

3. Phase 1 : Analyse et Conception

- 3.1 Choix du sujet d'analyse : Analyse des retours produits
- 3.2 Fixation des KPIs
- 3.3 Inspection des données sources
- 3.4 Modélisation conceptuelle du Data Warehouse
- 3.5 Conception du schéma en étoile
- 3.6 Identification des dimensions
- 3.7 Identification de la table de faits (FACT_RETOURS)

4. Phase 2 : ETL et Data Warehouse

- 4.1 Mise en place de l'environnement technique
- 4.2 Extraction des données
- 4.3 Transformation des données (Talend)
- 4.4 Script de creation du datawarehouse
- 4.5 Création des jobs ETL
- 4.6 Validation des données

5. Phase 3 : Analyse et Visualisation

- 5.1 Création des tableaux de bord
- 5.2 Analyse des KPIs
- 5.3 Visualisation des retours produits
- 5.4 Analyse des performances (produits, clients, magasins)
- 5.5 Decision

6. Phase 4 : Machine Learning

- 6.1 Préparation des données
- 6.2 Arbre de décision
- 6.3 Réseaux de neurones (R Studio)
- 6.4 Évaluation et comparaison des modèles
- 6.5 Prédiction des retours produits

7. Conclusion

8. Annexe

1. Objectif du Projet

1.1 Contexte du projet

Dans un environnement de vente au détail multi-magasins, les retours produits représentent un enjeu opérationnel et financier majeur. Chaque retour engendre un coût logistique, un remboursement client et une perte potentielle de revenus. Comprendre les causes et les tendances des retours est essentiel pour améliorer la satisfaction client et réduire les pertes.

Ce projet s'inscrit dans le cadre du module de Business Intelligence et vise à mettre en pratique l'ensemble de la chaîne analytique : de la modélisation du Data Warehouse jusqu'à la prédiction par Machine Learning.

1.2 Objectif principal

L'objectif principal est d'analyser les retours produits dans un réseau retail multi-magasins afin d'identifier les catégories de produits, les magasins et les profils clients les plus impactés, et de prédire si une commande sera retournée.

1.3 Bénéfices attendus

- Réduire le taux de retour global en identifiant les causes racines
- Optimiser la politique de remboursement par catégorie de produit
- Prioriser les actions correctives sur les magasins avec le plus de retours
- Anticiper les retours grâce à un modèle prédictif de Machine Learning
- Offrir aux décideurs un tableau de bord interactif et actualisable

2. Présentation des Données

2.1 Nom du dataset

Sales Retail Dataset — Jeu de données de vente au détail multi-magasins.

2.2 Description brève du dataset

Le dataset comprend 12 tables relationnelles couvrant l'ensemble des opérations d'une entreprise retail : clients, commandes, produits, paiements, expéditions et retours. Il contient 300 000 commandes, 600 000 lignes de commande et 30 000 retours sur la période 2020–2024.

2.3 Problématique métier visée

Quels produits, catégories et magasins génèrent le plus de retours ? Peut-on prédire si une commande sera retournée afin d'agir de manière proactive ?

2.4 Relations entre les tables

Table	Clé Primaire	Reliée à	Via Clé Étrangère
returns	return_id	order_items	order_item_id
order_items	order_item_id	orders, products	order_id, product_id
orders	order_id	customers, stores, promotions	customer_id, store_id, promotion_id
products	product_id	categories, suppliers	category_id, supplier_id
customers	customer_id	orders	customer_id
stores	store_id	orders	store_id
categories	category_id	products	category_id

3. Phase 1 : Analyse et Conception

3.1 Choix du sujet d'analyse : Analyse des retours produits

Le sujet choisi est l'**analyse des retours produits**. L'analyse permet d'identifier les leviers d'action pour réduire ce coût.

3.2 Fixation des KPIs

#	KPI	Description	Formule
KPI 1	Taux de retour par catégorie	% de lignes retournées par catégorie produit	$\text{Nb retours catégorie} / \text{Nb lignes catégorie} \times 100$
KPI 2	Montant total remboursé	Somme de tous les remboursements accordés	$\Sigma \text{ refund_amount}$
KPI 3	Taux de retour par magasin	% de retours par point de vente	$\text{Nb retours magasin} / \text{Nb lignes magasin} \times 100$
KPI 4	Produit avec le plus de retours	Produit ayant le plus grand nombre de retours	$\text{MAX}(\text{COUNT retours GROUP BY product_id})$
KPI 5	Montant moyen de remboursement	Remboursement moyen par retour	$\Sigma \text{ refund_amount} / \text{Nb retours}$
KPI 6	Perte estimée totale	Évaluation financière globale du manque à gagner brut causé par les produits retournés.	$\Sigma (\text{qty_returned} \times \text{unit_price})$
KPI 7	Taux de retour par ancienneté client	% de retours selon l'année d'inscription du client — permet d'identifier si les nouveaux clients retournent plus que les anciens	$\text{Nb retours par signup_year} / \text{Nb total retours} \times 100$

3.3 Inspection des données sources :

Les attributs (colonnes) les plus pertinents des fichiers sources pour le sujet "Analyse des retours produits" sont inspectés ci-dessous selon les critères de Qualité (0–3) et d'Utilité (0–3).

Nom du fichier : returns.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
return_id	Identifiant unique du retour	Int	3	3	
order_item_id	Clé étrangère vers order_items	Int	3	3	
refund	Montant remboursé	Double	2	3	Présence de valeurs incohérentes sur des lignes à corriger dans Talend.

Nom du fichier : order_items.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
order_item_id	Identifiant unique de la ligne de commande	Int	3	3	
order_id	Clé étrangère vers orders	Int	3	3	
product_id	Clé étrangère vers products	Int	3	3	
qty	Quantité d'articles commandés	Int	3	3	
price	Prix unitaire du produit	Double	3	3	

Nom du fichier : orders.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
order_id	Identifiant unique de la commande	Int	3	3	
customer_id	Identifiant du client ayant commandé	Int	3	3	
store_id	Identifiant du magasin (point de vente)	Int	3	3	
order_date	Date à laquelle la commande a été passée	Date	2	3	Type texte dans le CSV, nécessite l'utilisation de <code>TalendDate.parseDate</code> .
promotion_id	Identifiant de la promotion appliquée	Int	3	3	

Nom du fichier : products.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
product_id	Identifiant unique du produit	Int	3	3	
category_id	Identifiant de la catégorie rattachée	Int	3	3	
price	Prix catalogue d'origine du produit	Double	3	3	

Nom du fichier :categories.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
category_id	Identifiant unique de la catégorie	Int	3	3	
category_name	Libellé textuel de la catégorie	String	3	3	

Nom du fichier :stores.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
store_id	Identifiant unique du magasin	Int	3	3	
city	Ville où est implanté le magasin	String	3	3	

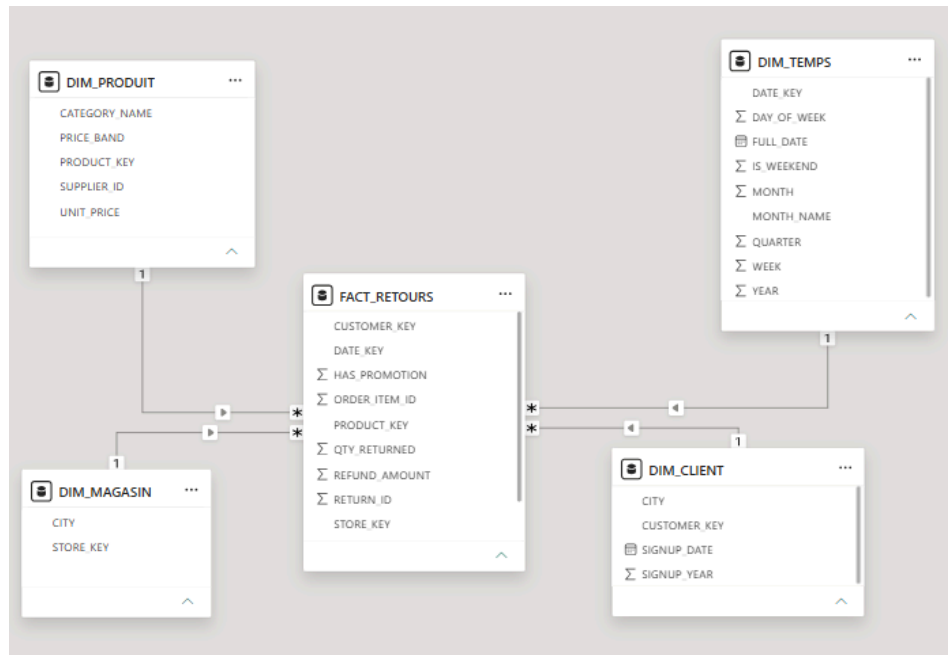
Nom du fichier :customers.csv

Nom	Description	Type	Qualité	Utilité	Commentaire
customer_id	Identifiant unique du client	Int	3	3	
city	Ville de résidence du client	String	3	2	Utilité notée 2 : non indispensable pour la table de faits mais enrichit la dimension.
signup_date	Date d'inscription du client	Date	2	3	Qualité notée 2 : nécessite un parsing de date pour en extraire l'année en format entier.

3.4 Modélisation conceptuelle du Data Warehouse

L'architecture retenue est le schéma en étoile (Star Schema). Ce choix est justifié par : La simplicité des requêtes analytiques (moins de jointures qu'un schéma flocon) , Les meilleures performances en lecture pour les outils BI (Power BI) et La clarté du modèle pour la présentation et la maintenance .

3.5 Conception du schéma en étoile :



3.6 Identification des dimensions :

Dimension	Clé	Attributs principaux	Source
DIM_TEMPS	date_key (NUMBER YYYYMMDD)	year, quarter, month, month_name, week, day_of_week, is_weekend	orders.order_date — généré
DIM_PRODUT	product_key (NUMBER)	category_name, supplier_id, unit_price, price_band	products + categories
DIM_MAGASIN	store_key (NUMBER)	store_id, city	stores
DIM_CLIENT	customer_key (NUMBER)	customer_id, city, signup_date, signup_year	customers

3.7 Identification de la table de faits (FACT_RETOURS) :

Granularité : la ligne de retour d'un produit (invoice line / ligne facture).
Cela signifie que chaque enregistrement dans **FACT_RETOURS** correspond exactement à **un produit retourné au sein d'une transaction de retour**.
Total : 30 000 lignes.

Colonne	Type Oracle	Source	Description
return_id (PK)	NUMBER	returns.return_id	Clé primaire naturelle — identifiant unique du retour
order_item_id (DD)	NUMBER	order_items.order_item_id	Dimension dégénérée — traçabilité de la ligne de commande retournée
date_key (FK)	NUMBER	orders.order_date	Lien vers DIM_TEMPS
product_key (FK)	NUMBER	products.product_id	Lien vers DIM_PRODUIT
store_key (FK)	NUMBER	stores.store_id	Lien vers DIM_MAGASIN
customer_key (FK)	NUMBER	customers.customer_id	Lien vers DIM_CLIENT
refund_amount	NUMBER(10,2)	returns.refund	Mesure Additive : Montant remboursé
qty_returned	NUMBER	order_items.qty	Mesure Additive : Quantité retournée
unit_price	NUMBER(10,2)	order_items.price	Mesure Non Additive : Prix unitaire du produit
total_lost	NUMBER(10,2)	Calculé	Mesure Additive qty_returned × unit_price (la perte financière brute)
has_promotion	NUMBER(1)	orders.promotion_id	Mesure Non Additive 1 si commande avec promotion, 0 sinon

4. Phase 2 : ETL et Data Warehouse

4.1 Mise en place de l'environnement technique

Composant	Technologie	Détail
Base source	Fichiers CSV	12 fichiers, encodage UTF-8, séparateur virgule
Base cible (DWH)	Oracle SQL Developer	Oracle Database 19c, schéma DWH_RETOURS
Outil ETL	Talend Open Studio for Data Integration	Version 8.x, driver ojdbc8.jar
Visualisation	Power BI Desktop	Connecteur Oracle natif
Machine Learning	R Studio	R 4.3, packages : rpart, neuralnet, caret

4.2 Extraction des données

Les données sont extraites depuis les fichiers CSV sources. Dans Talend, le composant tFileInputDelimited est utilisé pour chaque fichier. Les fichiers utilisés pour le sujet Retours produits sont :

- returns.csv — table principale (30 000 lignes)
- order_items.csv — lignes de commande (600 000 lignes)
- orders.csv — commandes avec dates, magasins et promotions (300 000 lignes)
- products.csv — informations produits avec prix (10 000 lignes)
- categories.csv — catégories de produits (30 lignes)
- customers.csv — informations clients (50 000 lignes)
- stores.csv — informations magasins (100 lignes)

4.3 Transformation des données (Talend)

La matrice de transformation décrit les règles appliquées à chaque colonne lors du passage des sources vers le Data Warehouse.

Champs cible	Table cible	Champs source	Table source	Règle de transformation
date_key	DIM_TEMPS	order_date	orders	Integer.parseInt(TalendDate.formatDate("yyyyMMdd", order_date))
year	DIM_TEMPS	order_date	orders	TalendDate.getPartOfDate("YEAR", order_date)
quarter	DIM_TEMPS	order_date	orders	TalendDate.getPartOfDate("QUARTER", order_date)
month	DIM_TEMPS	order_date	orders	TalendDate.getPartOfDate("MONTH", order_date)
month_name	DIM_TEMPS	order_date	orders	TalendDate.formatDate("MMMM", order_date)
week	DIM_TEMPS	order_date	orders	TalendDate.getPartOfDate("WEEK", order_date)
day_of_week	DIM_TEMPS	order_date	orders	TalendDate.getPartOfDate("DAY_OF_WEEK", order_date)
is_weekend	DIM_TEMPS	order_date	orders	(TalendDate.getPartOfDate("DAY_OF_WEEK", order_date)==1 TalendDate.getPartOfDate("DAY_OF_WEEK", order_date)==7) ? 1 : 0
product_key	DIM_PRODUCT	product_id	products	Direct (clé naturelle)
category_name	DIM_PRODUCT	category_name	categories	JOIN sur category_id via tMap
supplier_id	DIM_PRODUCT	supplier_id	products	Direct
unit_price	DIM_PRODUCT	price	products	Direct

price_band	DIM_PRODUCT	price	products	(row1.price < 1000) ? "Low" : (row1.price < 3000) ? "Mid" : "High"
store_key	DIM_STORE	store_id	stores	Direct (clé naturelle)
city	DIM_STORE	city	stores	Direct
customer_key	DIM_CUSTOMER	customer_id	customers	Direct (clé naturelle)
city	DIM_CUSTOMER	city	customers	Direct
signup_date	DIM_CUSTOMER	signup_date	customers	Direct
signup_year	DIM_CUSTOMER	signup_date	customers	EXTRACT(YEAR FROM signup_date)
return_id	FACT_RETURNS	return_id	returns	Direct
refund_amount	FACT_RETURNS	refund	returns	Direct
qty_returned	FACT_RETURNS	qty	order_items	Direct
unit_price	FACT_RETURNS	price	order_items	Direct
total_lost	FACT_RETURNS	qty, price	order_items	qty × price (tMap : row1.qty * row1.price)
has_promotion	FACT_RETURNS	promotion_id	orders	(row3.promotion_id != null) ? 1 : 0

4.4 Script Oracle de creation du schema en etoile :

```
-- CREATION DU DATA WAREHOUSE : SCHEMA EN ETOILE
-- DIM_TEMPS
CREATE TABLE DIM_TEMPS (
    date_key      NUMBER PRIMARY KEY,
    full_date     DATE,
    year          NUMBER,
    quarter       NUMBER,
    month         NUMBER,
    month_name    VARCHAR2(20),
    week          NUMBER,
    day_of_week   NUMBER,
    is_weekend    NUMBER(1)
);
```

```
-- DIM_PRODUIT
CREATE TABLE DIM_PRODUIT (
    product_key   NUMBER PRIMARY KEY,
    category_name VARCHAR2(50),
    supplier_id   NUMBER,
    unit_price    NUMBER(10,2),
    price_band    VARCHAR2(10)
);
```

```
-- DIM_MAGASIN
CREATE TABLE DIM_MAGASIN (
    store_key     NUMBER PRIMARY KEY,
    city          VARCHAR2(50)
);
-- DIM_CLIENT
CREATE TABLE DIM_CLIENT (
    customer_key  NUMBER PRIMARY KEY,
```

```
city          VARCHAR2(50),
signup_date   DATE,
signup_year   NUMBER
);
```

```
-- FACT_RETOURS
CREATE TABLE FACT_RETOURS (
    return_id      NUMBER,
    order_item_id  NUMBER,          -- DD : traçabilité grain
    product_key    NUMBER,
    store_key      NUMBER,
    customer_key   NUMBER,
    date_key       NUMBER,
    refund_amount  NUMBER(10,2),
    qty_returned   NUMBER,
    unit_price     NUMBER(10,2),
    total_lost     NUMBER(10,2),
    has_promotion  NUMBER(1) DEFAULT 0,

    CONSTRAINT pk_fact_retours PRIMARY KEY (return_id),
    CONSTRAINT fk_date          FOREIGN KEY (date_key)
REFERENCES DIM_TEMPS(date_key),
    CONSTRAINT fk_product      FOREIGN KEY (product_key)
REFERENCES DIM_PRODUIT(product_key),
    CONSTRAINT fk_store        FOREIGN KEY (store_key)
REFERENCES DIM_MAGASIN(store_key),
    CONSTRAINT fk_customer     FOREIGN KEY (customer_key)
REFERENCES DIM_CLIENT(customer_key)
);
```

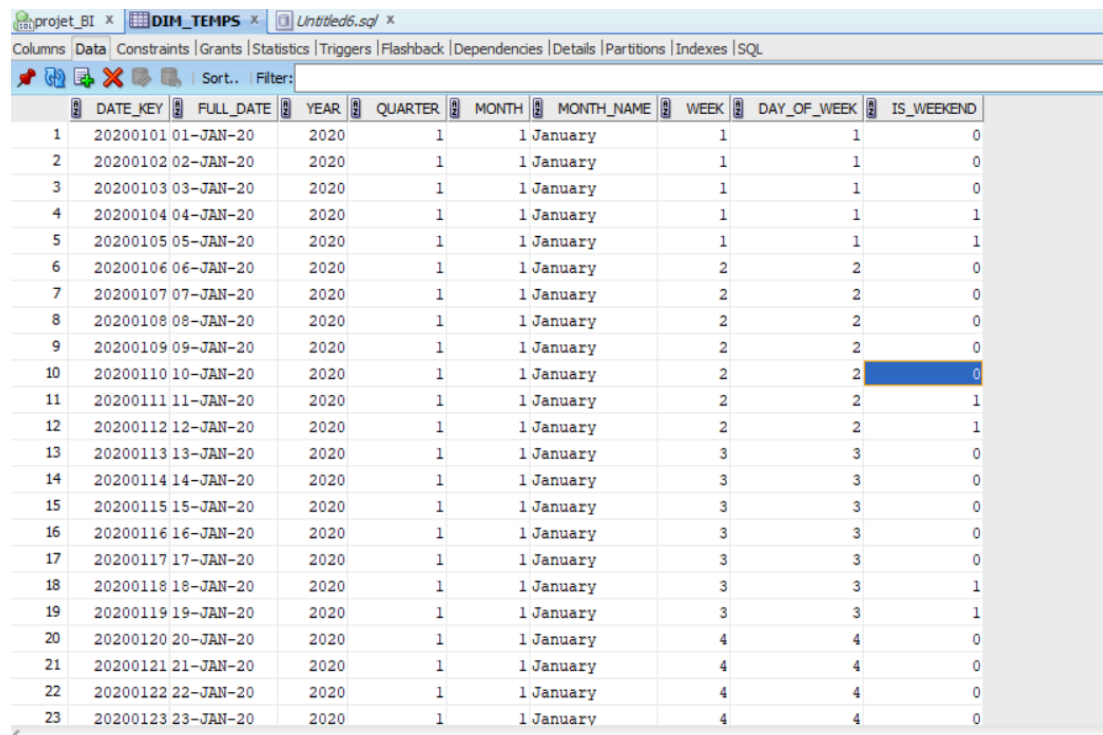

4.5 Création des jobs ETL

Job 1 — Chargement DIM_TEMPS

Ce job génère le calendrier complet de 2020 à 2024 et le charge dans DIM_TEMPS.

Composant Talend	Paramètres	Rôle
tRowGenerator	1827 lignes, colonne <code>id_sequence</code> de type Integer, fonction <code>Numeric.sequence("s1", 1, 1)</code>	Génération d'une séquence numérique de 1 à 1827 représentant chaque jour
tMap	Calcul des colonnes à partir de <code>row2.id_sequence</code> via <code>TalendDate.addDate</code>	Enrichissement des colonnes temporelles à partir de la séquence
tOracleOutput	Table: <code>DIM_TEMPS</code> , Action on data: <code>Insert</code> , Connection Type: Oracle SID, SID: <code>XE</code>	Chargement dans Oracle

resultat de job 1: Le calendrier généré couvre la période 2020–2024 avec 1827 jours. Chaque ligne représente un jour avec ses attributs temporels .



	DATE_KEY	FULL_DATE	YEAR	QUARTER	MONTH	MONTH_NAME	WEEK	DAY_OF_WEEK	IS_WEEKEND
1	20200101	01-JAN-20	2020	1	1	January	1	1	0
2	20200102	02-JAN-20	2020	1	1	January	1	1	0
3	20200103	03-JAN-20	2020	1	1	January	1	1	0
4	20200104	04-JAN-20	2020	1	1	January	1	1	1
5	20200105	05-JAN-20	2020	1	1	January	1	1	1
6	20200106	06-JAN-20	2020	1	1	January	2	2	0
7	20200107	07-JAN-20	2020	1	1	January	2	2	0
8	20200108	08-JAN-20	2020	1	1	January	2	2	0
9	20200109	09-JAN-20	2020	1	1	January	2	2	0
10	20200110	10-JAN-20	2020	1	1	January	2	2	0
11	20200111	11-JAN-20	2020	1	1	January	2	2	1
12	20200112	12-JAN-20	2020	1	1	January	2	2	1
13	20200113	13-JAN-20	2020	1	1	January	3	3	0
14	20200114	14-JAN-20	2020	1	1	January	3	3	0
15	20200115	15-JAN-20	2020	1	1	January	3	3	0
16	20200116	16-JAN-20	2020	1	1	January	3	3	0
17	20200117	17-JAN-20	2020	1	1	January	3	3	0
18	20200118	18-JAN-20	2020	1	1	January	3	3	1
19	20200119	19-JAN-20	2020	1	1	January	3	3	1
20	20200120	20-JAN-20	2020	1	1	January	4	4	0
21	20200121	21-JAN-20	2020	1	1	January	4	4	0
22	20200122	22-JAN-20	2020	1	1	January	4	4	0
23	20200123	23-JAN-20	2020	1	1	January	4	4	0

Job 2 — Chargement DIM_PRODUIT et DIM_MAGASIN

Composant Talend	Paramètres	Rôle
tFileInputExcel	products.csv, Header: oui, Separator: ,	Lecture fichier produits
tFileInputExcel	categories.csv, Header: oui, Separator: ,	Lecture fichier catégories
tMap	JOIN products + categories sur category_id, calcul price_band	Enrichissement dimension produit
tSortRow	Critère : PRODUCT_KEY , Ordre : Ascendant	Tri des produits par clé produit
tOracleOutput	Table: DIM_PRODUIT, Action: INSERT	Chargement produits dans Oracle
tFileInputExcel	stores.csv, Header: oui, Separator: ,	Lecture fichier magasins
tOracleOutput	Table: DIM_MAGASIN, Action: INSERT	Chargement magasins dans Oracle

resultat de job 2: Dim_produit : dim_magazin :

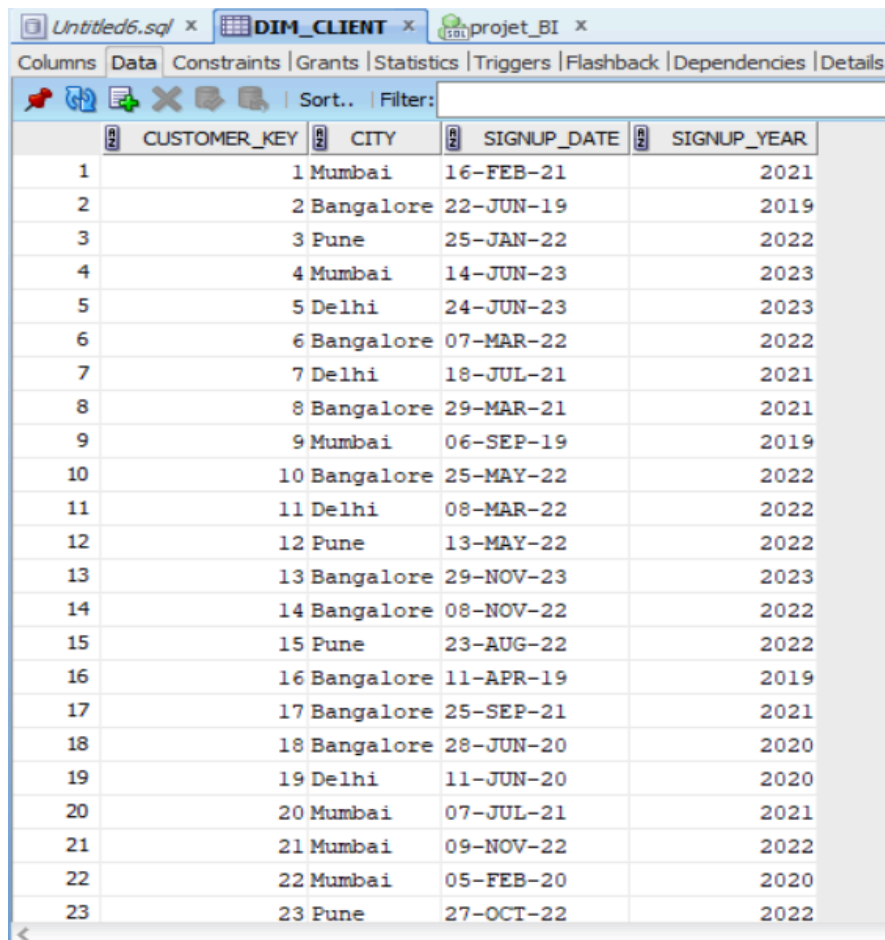
PRODUCT_KEY	CATEGORY_NAME	SUPPLIER_ID	UNIT_PRICE	PRICE_BAND
1	1 Cat_9	135	3987	High
2	2 Cat_18	194	4412	High
3	3 Cat_23	118	3548	High
4	4 Cat_19	139	1971	Mid
5	5 Cat_10	115	4495	High
6	6 Cat_17	53	2634	Mid
7	7 Cat_8	160	3028	High
8	8 Cat_30	21	3686	High
9	9 Cat_29	74	1844	Mid
10	10 Cat_29	77	316	Low
11	11 Cat_9	61	564	Low
12	12 Cat_14	146	3422	High
13	13 Cat_30	17	1910	Mid
14	14 Cat_10	187	2403	Mid
15	15 Cat_19	145	3155	High
16	16 Cat_10	122	3332	High
17	17 Cat_15	124	1578	Mid
18	18 Cat_7	42	4115	High
19	19 Cat_8	63	935	Low
20	20 Cat_27	114	616	Low
21	21 Cat_6	33	3951	High
22	22 Cat_10	140	2753	Mid
23	23 Cat_19	46	1674	Mid

STORE_KEY	CITY
1	Pune
2	Pune
3	Delhi
4	Mumbai
5	Mumbai
6	Mumbai
7	Delhi
8	Bangalore
9	Delhi
10	Bangalore
11	Delhi
12	Pune
13	Bangalore
14	Mumbai
15	Pune
16	Pune
17	Delhi
18	Bangalore
19	Mumbai
20	Mumbai
21	Delhi
22	Mumbai
23	Pune
24	Pune
25	Delhi
26	Pune
27	Bangalore
28	Mumbai
29	Bangalore
30	Pune

Job 3 — Chargement DIM_CLIENT

Composant Talend	Paramètres	Rôle
tFileInputExcel	customers.csv, Header: oui, Separator: ,	Lecture fichier clients
tUniqRow	Clé : customer_id , Case Sensitive : Non	Suppression des doublons sur les clients
tSortRow	Critère : customer_id , Ordre : Ascendant	Tri des clients par identifiant
tMap	Calcul signup_year = GetPartOfDate(YEAR FROM signup_date)	Ajout de l'année d'inscription
tOracleOutput	Table: DIM_CLIENT, Action: INSERT	Chargement clients dans Oracle

resultat de job 3:



	CUSTOMER_KEY	CITY	SIGNUP_DATE	SIGNUP_YEAR
1	1	Mumbai	16-FEB-21	2021
2	2	Bangalore	22-JUN-19	2019
3	3	Pune	25-JAN-22	2022
4	4	Mumbai	14-JUN-23	2023
5	5	Delhi	24-JUN-23	2023
6	6	Bangalore	07-MAR-22	2022
7	7	Delhi	18-JUL-21	2021
8	8	Bangalore	29-MAR-21	2021
9	9	Mumbai	06-SEP-19	2019
10	10	Bangalore	25-MAY-22	2022
11	11	Delhi	08-MAR-22	2022
12	12	Pune	13-MAY-22	2022
13	13	Bangalore	29-NOV-23	2023
14	14	Bangalore	08-NOV-22	2022
15	15	Pune	23-AUG-22	2022
16	16	Bangalore	11-APR-19	2019
17	17	Bangalore	25-SEP-21	2021
18	18	Bangalore	28-JUN-20	2020
19	19	Delhi	11-JUN-20	2020
20	20	Mumbai	07-JUL-21	2021
21	21	Mumbai	09-NOV-22	2022
22	22	Mumbai	05-FEB-20	2020
23	23	Pune	27-OCT-22	2022

Job 4 — Chargement FACT_RETOURS (Job principal)

Ce job assemble toutes les sources pour construire la table de faits centrale.

Composant Talend	Paramètres	Rôle
tFileInputExcel	Fichier <code>returns.xlsx</code>	Lecture des données de retours
tFileInputDelimited	Fichier <code>order_items</code>	Lecture des lignes de commande
tMap (Jointure 1)	JoinJointure sur <code>order_item_id</code> ture sur <code>product_id</code>	Association des retours aux lignes de commande
tFileInputDelimited	<code>orders</code>	Lecture des commandes
tFileInputExcel	<code>products</code>	Lecture des produits
tMap	Jointures avec <code>orders</code> et <code>products</code> et calcul : <code>total_lost = qty * price</code> , <code>has_promotion = CASE WHEN promotion_id IS NOT NULL</code>	Ajout des informations de commande, client, magasin et produit et Calcul des ccolonnes dérivées
tSortRow	Tri sur <code>return_id</code> (ASC)	Ordonnancement des données avant chargement
tOracleOutput	Table: <code>FACT_RETOURS</code> , Action: <code>INSERT</code> ,	Chargement optimisé dans Oracle

Remarque : Les fichiers volumineux (> 60 000 lignes) ont été chargés avec **tFileInputDelimited** car **tFileInputExcel** présente des limitations de performance pour le traitement de grands volumes de données.

resultat de job 4:

Untitled6.sql x projct_BI x FACT_RETOURS x Columns Data Constraints Grants Statistics Triggers Flashback Dependencies Details Partitions Indexes SQL Sort.. Filter:												
	RETI...	ORDER_I...	PRODUCT_KEY	STORE_...	CUSTOMER_...	DATE_KEY	REFUND_AMO...	QTY...	UNIT_PRICE	TOTAL_LOST	HAS...	
1	1	496351	1996	18	41219	20210225	2282	3	4592	13776	1	
2	2	522631	45	8	10487	20221117	4637	1	929	929	1	
3	3	481424	4410	51	15714	20221122	2421	3	1797	5391	1	
4	4	207559	5903	15	45922	20230529	1805	4	2103	8412	1	
5	5	265929	4329	52	21443	20201016	4907	3	4623	13869	1	
6	6	179208	4207	44	43192	20210427	88	3	4566	13698	1	
7	7	472887	8990	77	35851	20230101	4640	2	1714	3428	1	
8	8	103776	7800	56	19897	20230529	3627	3	4917	14751	1	
9	9	137189	8463	96	44872	20220203	1100	1	3623	3623	1	
10	10	393105	9818	56	1922	20200402	2716	3	4817	14451	1	
11	11	372056	2049	42	33438	20210206	4627	1	1846	1846	1	
12	12	515705	7253	45	16133	20220426	3573	3	2601	7803	1	
13	13	51668	5266	63	27130	20200711	384	2	2902	5804	1	
14	14	111775	3844	90	3558	20211110	2401	2	500	1000	1	
15	15	179452	1153	27	38172	20210315	2088	4	4201	16804	1	
16	16	59971	3897	60	42449	20200312	2625	1	2100	2100	1	
17	17	409618	4707	49	9828	20201019	2044	1	140	140	1	
18	18	331870	8179	85	13218	20221212	4404	4	1357	5428	1	
19	19	599855	6753	75	29233	20230312	1402	4	1300	5200	1	
20	20	467400	4702	62	30723	20220830	3336	1	1662	1662	1	
21	21	488808	3043	26	14587	20200716	2730	1	3889	3889	1	
22	22	351599	4284	60	21693	20211027	1782	3	3240	9720	1	
23	23	287231	643	66	11583	20210512	1217	3	4228	12684	1	
24	24	573505	1386	11	28836	20210721	1211	2	3384	6768	1	
25	25	169331	9481	73	20737	20201027	3579	4	438	1752	1	
26	26	378680	1812	35	44803	20220404	4603	3	4933	14799	1	
27	27	49997	5607	1	28001	20220611	884	1	4324	4324	1	
28	28	5539	8570	12	13511	20211118	3214	2	1100	2200	1	
29	29	536618	7161	59	49074	20230924	2648	2	951	1902	1	
30	30	395110	2113	45	35303	20210207	2049	4	2328	9312	1	

4.6 Validation des données

Après chargement, les contrôles suivants sont exécutés dans Oracle SQL Developer :

Contrôle	Requête Oracle	Résultat attendu
Comptage FACT_RETOURS	SELECT COUNT(*) FROM FACT_RETOURS	30 000
Clés orphelines produit	SELECT COUNT(*) FROM FACT_RETOURS f LEFT JOIN DIM_PRODUIT d ON f.product_key=d.product_key WHERE d.product_key IS NULL	0
Clés orphelines magasin	SELECT COUNT(*) FROM FACT_RETOURS f LEFT JOIN DIM_MAGASIN m ON f.store_key=m.store_key WHERE m.store_key IS NULL	0
Montants négatifs	SELECT COUNT(*) FROM FACT_RETOURS WHERE refund_amount < 0	0
Vérification montant total	SELECT SUM(refund_amount) FROM FACT_RETOURS	75 962 300

5. Phase 3 : Analyse et Visualisation

5.1 Création des tableaux de bord

Le tableau de bord est développé dans **Power BI Desktop**, connecté au Data Warehouse via les fichiers CSV exportés depuis Oracle SQL Developer (mode Import). Les relations sont définies automatiquement entre **FACT_RETOURS** et chaque dimension (cardinalité N:1, direction du filtre unique).

5.2 Analyse des KPIs

Les mesures DAX suivantes ont été créées dans Power BI pour calculer les indicateurs clés de performance :

```
Nb_Retours = COUNTROWS (FACT_RETOURS)
Montant_Total = SUM(FACT_RETOURS[refund_amount])
Montant_Moyen = AVERAGE (FACT_RETOURS[refund_amount])
Taux_Retour = DIVIDE ([Nb_Retours], 600000) * 100
Total Manque à Gagner = SUM(FACT_RETOURS[total_lost])
Taux_Retour_Cat = DIVIDE ([Nb_Retours], 30000, 0)
Taux_Retour_Store = DIVIDE ([Nb_Retours], 30000, 0)

Taux Retour Ancienneté % = DIVIDE (COUNTROWS (FACT_RETOURS),
CALCULATE (COUNTROWS (FACT_RETOURS), ALL (DIM_CLIENT))) * 100
```

Ces mesures s'adaptent automatiquement aux filtres appliqués par les segments interactifs (année, catégorie, ville, client).

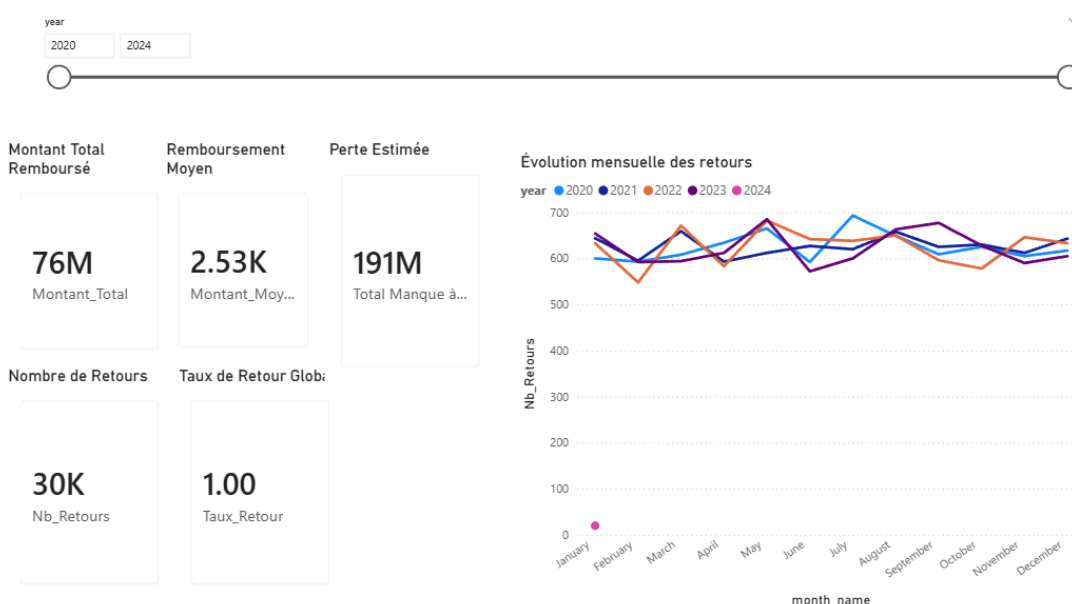
5.3 Visualisation des retours produits

Le tableau de bord est organisé en 4 pages distinctes, chacune dédiée à une dimension d'analyse :

Page	Titre	Visualisations	KPIs affichés
Page 1	Vue Générale	5 KPI Cards, Line chart retours par mois	Nb retours (30K), Montant (76M€), Moyenne (2,53K€), Taux retour, Perte Estimée (191M€)
Page 2	Par Catégorie	Bar chart taux retour par catégorie, Treemap montant remboursé	KPI 1 : Taux de retour par catégorie

Page 3	Par Magasin	Bar chart top 10 magasins, Donut chart, Table détail avec ville	KPI 3 : Taux de retour par magasin
Page 4	Par Produit	Table produit le plus retourné, Bar chart top 5, Table détail	KPI 4 : Produit avec le plus de retours
Page 5	Par Ancienneté Client	Histogramme taux de retour par signup_year, Table détail par cohorte	KPI 7 : Taux de retour par ancienneté client

• Page 1 :

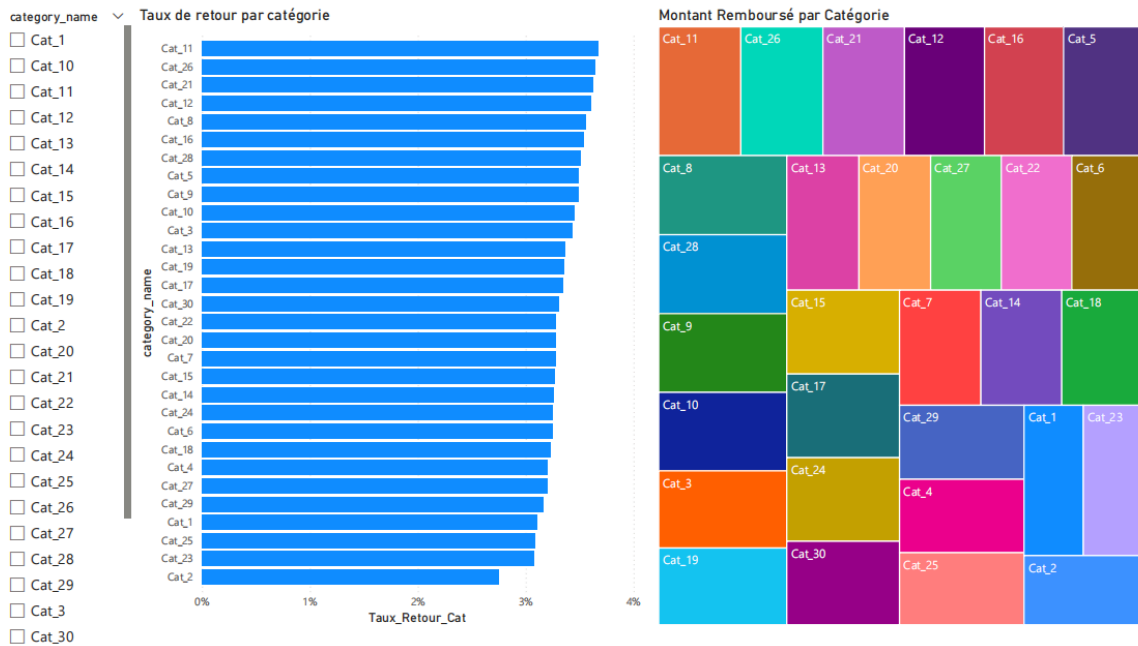


La page d'accueil présente les 5 indicateurs clés sous forme de cartes KPI :

- Nombre de retours : 30 000 retours sur 600 000 lignes de commande
- Montant total remboursé : 76 M
- Remboursement moyen par retour : 2 530
- Perte estimée (Manque à gagner) : 191 M

Le **graphique en courbes** illustre l'évolution mensuelle des retours par année (2020–2024), permettant d'identifier les tendances saisonnières et les pics de retours sur la période analysée.

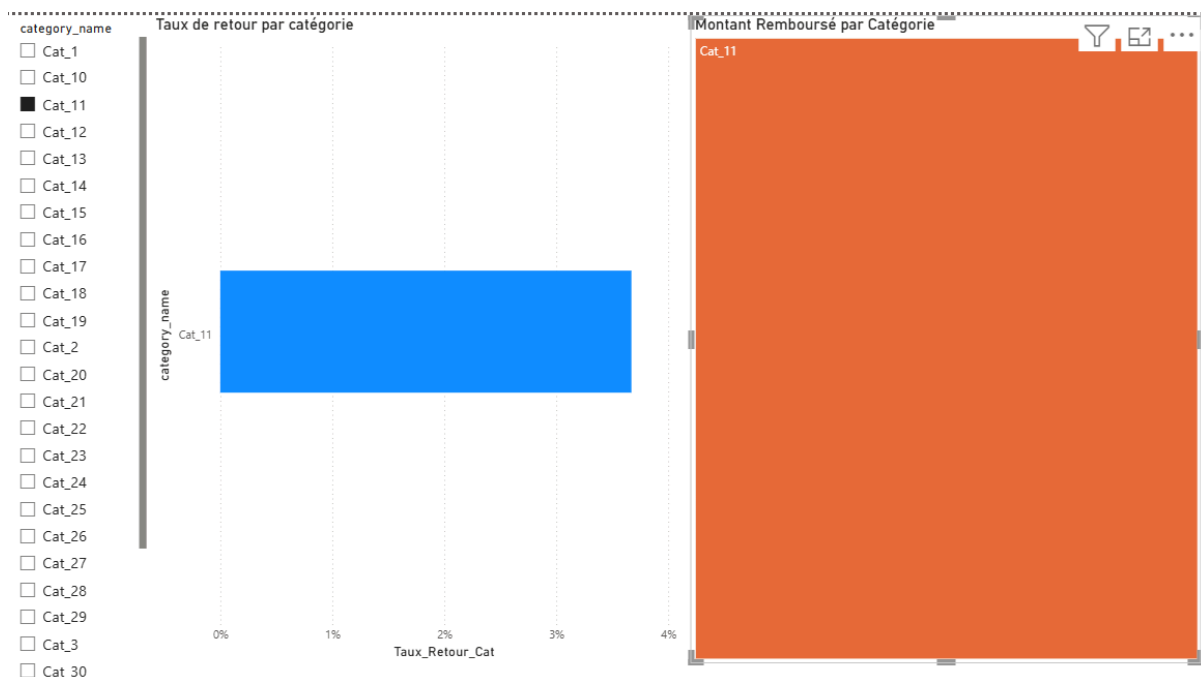
• Page 2 :



Le bar chart révèle que Cat_11 est la catégorie la plus retournée avec un taux de 3,67%, suivie de Cat_26 et Cat_21. Le Treemap confirme que Cat_11 concentre également le montant remboursé le plus élevé parmi les 30 catégories analysées.

Le segment interactif permet de filtrer les résultats par catégorie pour une analyse approfondie.

Par exemple catégorie 11 :



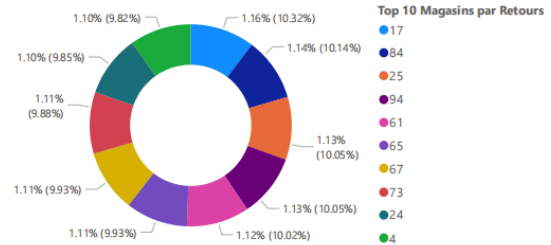
• Page 3 :

- city
- ☐ Bangalore
 - ☐ Delhi
 - ☐ Mumbai
 - ☐ Pune

Détail par Magasin

store_key	city	Nb_Retours	Montant_Total	Taux_Retour_Store
17	Delhi	347	904394	1.16%
84	Mumbai	341	852205	1.14%
25	Delhi	338	873780	1.13%
94	Pune	338	861834	1.13%
61	Pune	337	862398	1.12%
65	Pune	334	807817	1.11%
67	Pune	334	869442	1.11%
73	Bangalore	332	828407	1.11%
24	Pune	331	829898	1.10%
4	Mumbai	330	829208	1.10%
79	Bangalore	324	854735	1.08%
99	Bangalore	324	837172	1.08%
48	Mumbai	320	785709	1.07%
55	Mumbai	320	794328	1.07%
13	Bangalore	318	805034	1.06%
40	Bangalore	318	827301	1.06%
72	Pune	318	813513	1.06%
96	Pune	318	806599	1.06%
20	Mumbai	317	804885	1.06%
86	Mumbai	316	814741	1.05%
15	Pune	315	782750	1.05%
34	Mumbai	313	809243	1.04%
64	Bangalore	313	795483	1.04%
70	Delhi	313	804808	1.04%
5	Mumbai	312	770201	1.04%
23	Pune	312	787649	1.04%
19	Mumbai	311	750025	1.04%
51	Bangalore	311	783240	1.04%
62	Bangalore	311	808652	1.04%
47	Pune	310	796266	1.03%
93	Pune	310	806722	1.03%
Total		30000	75962300	100.00%

Taux_Retour_Store par store_key



Le top 10 des magasins montre que le magasin #17 de Delhi est le plus impacté avec 347 retours et un taux de 1,16%. Les villes de Delhi, Mumbai et Pune concentrent la majorité des retours.

Le tableau détaillé présente pour chaque magasin : le store_key, la ville, le nombre de retours, le montant total remboursé et le taux de retour. Le segment ville permet le filtrage interactif par localisation.

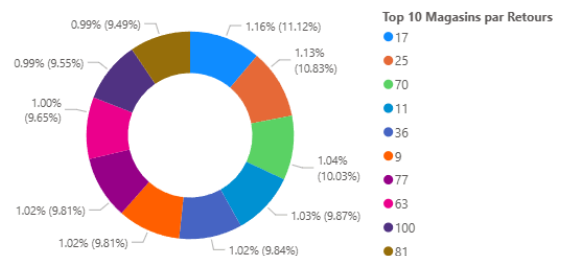
Par exemple les magasins de Delhi :

- city
- ☐ Bangalore
 - ☒ Delhi
 - ☐ Mumbai
 - ☐ Pune

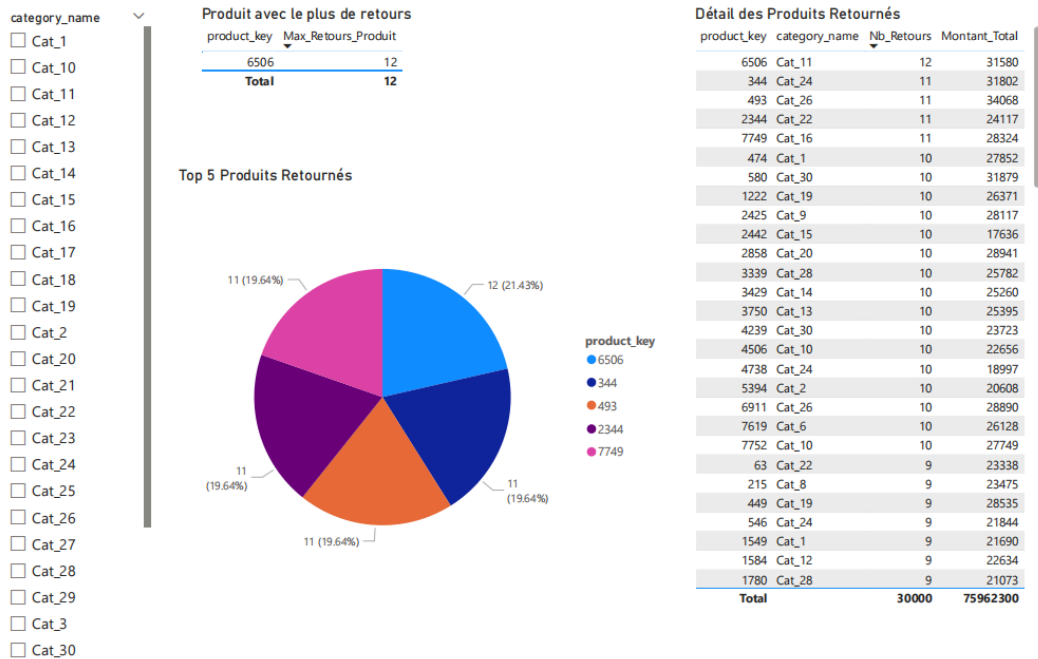
Détail par Magasin

store_key	city	Nb_Retours	Montant_Total	Taux_Retour_Store
17	Delhi	347	904394	1.16%
25	Delhi	338	873780	1.13%
70	Delhi	313	804808	1.04%
11	Delhi	308	760252	1.03%
36	Delhi	307	749683	1.02%
9	Delhi	306	745045	1.02%
77	Delhi	306	753416	1.02%
63	Delhi	301	774745	1.00%
100	Delhi	298	724911	0.99%
81	Delhi	296	723228	0.99%
45	Delhi	294	759880	0.98%
3	Delhi	289	758262	0.96%
7	Delhi	286	712877	0.95%
76	Delhi	286	731131	0.95%
44	Delhi	284	734662	0.95%
69	Delhi	280	713490	0.93%
21	Delhi	279	698855	0.93%
93	Delhi	279	719094	0.93%
71	Delhi	277	733223	0.92%
56	Delhi	274	708088	0.91%
46	Delhi	271	677959	0.90%
Total		6219	15761783	20.73%

Taux_Retour_Store par store_key



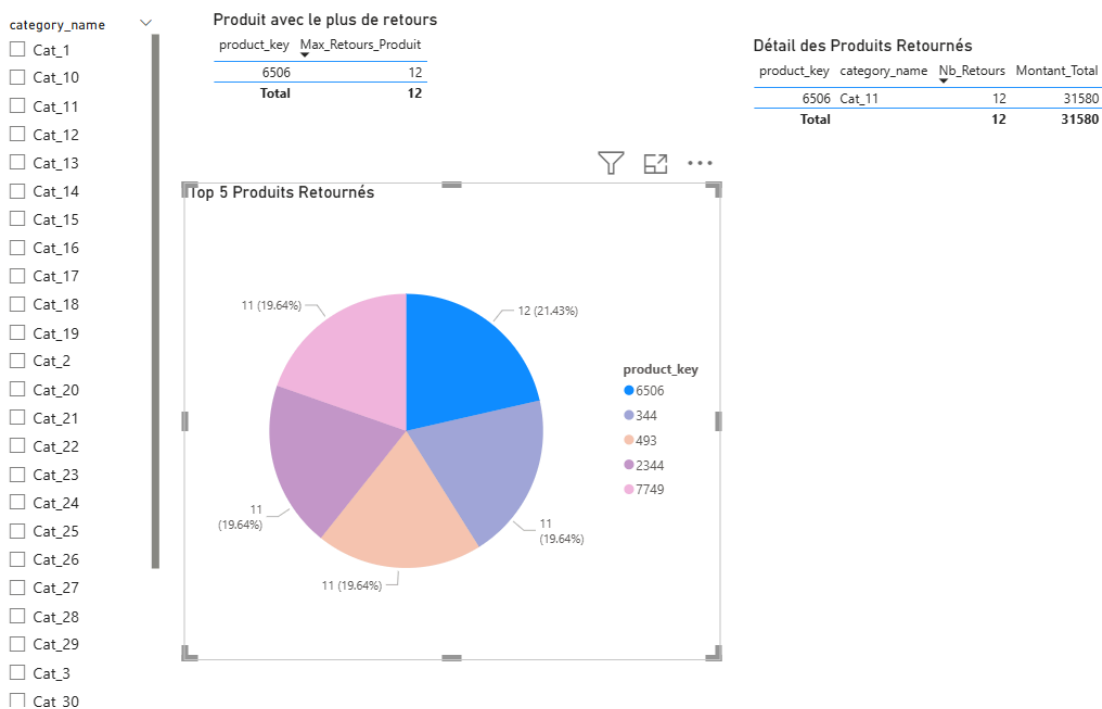
• Page 4 :



Le tableau révèle que le produit #6506 (catégorie Cat_11) est le produit le plus retourné avec 12 retours. Le top 5 regroupe les produits des catégories Cat_11, Cat_24, Cat_26, Cat_22 et Cat_16 avec des retours entre 11 et 12.

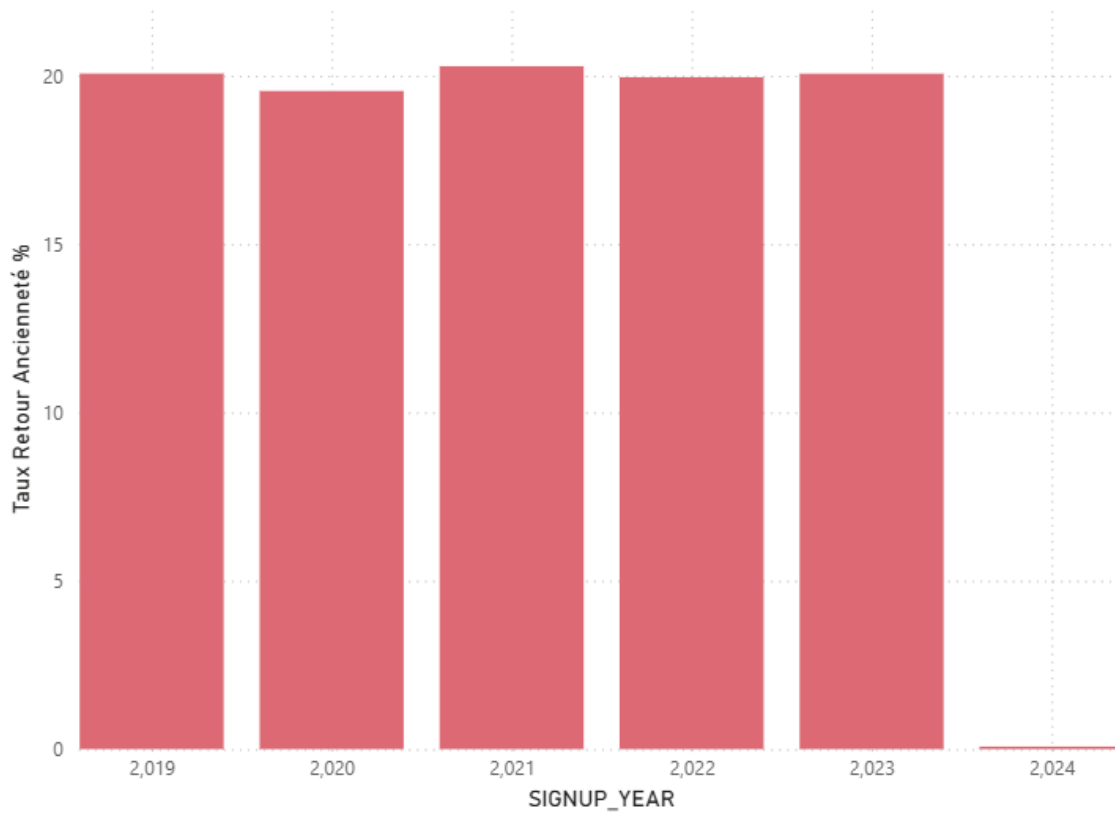
Le tableau détaillé des produits retournés permet d'identifier les catégories les plus impactées et les montants remboursés associés à chaque produit.

Par exemple le produit 6506 :



- **Page 5:**

Taux Retour Ancienneté % par SIGNUP_YEAR



Interprétation des résultats :

Le taux de retour est stable autour de 20% pour toutes les cohortes d'inscription (2019–2023), ce qui indique que l'ancienneté du client n'influence pas le comportement de retour. L'année 2024 représente 0,07% des retours en raison du faible nombre de clients inscrits sur cette période.

Ce KPI infirme l'hypothèse de départ "L'ancienneté client ne constitue pas un facteur différenciant dans le comportement de retour."

5.4 Analyse des performances (produits, clients, magasins):

KPI	Valeur calculée	Observation métier
KPI 1 — Catégorie la plus retournée	Cat_11 : 1 101 retours (3,67%)	Nécessite une investigation qualité sur les produits de la catégorie Cat_11. C'est la catégorie prioritaire pour les actions correctives.
KPI 2 — Montant total remboursé	75 962 300	Coût financier très significatif pour l'entreprise sur la période 2020–2024. Représente l'impact direct sur la trésorerie.
KPI 3 — Magasin le plus impacté	Magasin #17 Delhi : 347 retours (1,16%)	Peut indiquer un problème logistique ou de qualité local à Delhi. Une investigation sur site est recommandée.
KPI 4 — Produit le plus retourné	Produit #6506 : 12 retours (Cat_11)	Défaut potentiel ou mauvaise description produit. Action corrective prioritaire sur ce produit.
KPI 5 — Montant moyen remboursement	2 532	Confirme que les produits retournés sont de gamme élevée. L'impact unitaire de chaque retour est donc important.
KPI 6 — Perte estimée totale	191 M	Manque à gagner considérable sur 2020–2024. Souligne l'urgence de mettre en place des actions préventives.
KPI 7 _ Taux de retour par ancienneté client	~20% pour chaque cohorte (2019–2023)	L'ancienneté du client n'influence pas le comportement de retour. Le taux est uniforme sur toutes les années d'inscription, ce qui oriente les actions correctives vers d'autres leviers que la fidélisation client.

5.5 Decisions :

1. Action sur Cat_11 (KPI 1 + KPI 4)

- Auditer la qualité des produits de Cat_11
- Améliorer les descriptions produits en ligne
- Produit #6506 en priorité absolue — vérifier s'il présente un défaut de fabrication

2. Action sur le Magasin #17 Delhi (KPI 3)

- Investiguer les processus logistiques locaux
- Former le personnel sur la gestion des retours
- Vérifier la qualité de stockage et livraison à Delhi

3. Action financière (KPI 2 + KPI 5 + KPI 6)

- Revoir la politique de remboursement pour les produits High (>3000€)
- Mettre en place une inspection obligatoire avant remboursement pour les articles >2 532€
- Réduire le manque à gagner de 191M€ en ciblant les catégories à fort volume de retours

4. Politique client (KPI 7)

- L'ancienneté client n'étant pas un facteur de risque, concentrer les efforts de prévention sur le **produit** et le **magasin** plutôt que sur le profil client
- Abandonner les programmes de fidélisation comme levier anti-retour

6. Phase 4 : Machine Learning

6.1 Préparation des données

Problématique business : Prédire si un retour produit sera critique, c'est-à-dire générant un remboursement élevé avec une grande quantité retournée. Il s'agit d'un problème de classification binaire.

Variable cible : `retour_critique` (1 = retour critique, 0 = retour normal).

Distribution : 20% positifs / 80% négatifs, déséquilibre modéré géré par le paramétrage du modèle.

Feature	Type	Source	Justification
category_name	Catégoriel	DIM_PRODUIT	Certaines catégories génèrent structurellement des retours plus coûteux
price_band	Catégoriel	DIM_PRODUIT	Les produits chers (High) ont des remboursements plus élevés
unit_price	Numérique	FACT_RETOURS	Prix unitaire du produit — directement lié au montant remboursé
qty_returned	Numérique	FACT_RETOURS	Variable clé : plus la quantité est grande, plus le retour est critique
store_city	Catégoriel	DIM_MAGASIN	Certaines villes ont des comportements de retour différents
month	Numérique	DIM_TEMPS	Saisonnalité des retours critiques (pics après périodes de fêtes)
is_weekend	Binaire	DIM_TEMPS	Les achats du weekend sont plus impulsifs et génèrent plus de retours critiques
signup_year	Numérique	DIM_CLIENT	Les nouveaux clients retournent plus souvent et en plus grande quantité

6.2 Arbre de décision

L'arbre de décision (Decision Tree via rpart) est un algorithme interprétable qui segmente les données selon des règles logiques. Il est idéal comme modèle de base car ses décisions sont lisibles et explicables.

La variable cible retour_critique est construite comme une combinaison de deux critères : le montant de remboursement dans le top 40% et une quantité retournée supérieure ou égale à 3 articles. L'arbre est entraîné avec une profondeur maximale de 6 niveaux et un cp de 0.0005.

- **Code de l'algorithme Arbre de décision :**

```
# Arbre de décision
# Cible composite : retour CRITIQUE
# = remboursement élevé ET quantité élevée
# ===== ARBRE DE DÉCISION =====

library(rpart)
library(rpart.plot)
library(caret)

# Cible
df$retour_critique <- as.factor(ifelse(
  df$refund_amount >= quantile(df$refund_amount, 0.60) &
  df$qty_returned >= 3, 1, 0
))

# Conversion factors
df$has_promotion <- as.factor(df$has_promotion)
df$is_weekend <- as.factor(df$is_weekend)
df$category_name <- as.factor(df$category_name)
df$city <- as.factor(df$city)
df$price_band <- as.factor(df$price_band)
```



```

df_model <- df[, c("unit_price", "qty_returned",
                  "has_promotion",
                  "category_name", "price_band", "month",
                  "is_weekend", "signup_year", "city",
                  "retour_critique")]

df_model <- na.omit(df_model)

set.seed(42)

idx <- createDataPartition(df_model$retour_critique, p =
0.8, list = FALSE)
train <- df_model[idx, ]
test <- df_model[-idx, ]

# Modèle
model_dt <- rpart(retour_critique ~ ., data = train,
method = "class",
                  control = rpart.control(maxdepth = 6, cp
= 0.0005,
                                          minsplit = 50,
minbucket = 20))

# Visualisation
png("arbre_final.png", width = 6000, height = 4000, res =
200)
rpart.plot(model_dt,
            type = 4,
            extra = 0,
            fallen.leaves = TRUE,
            cex = 0.75,
            tweak = 1.3,

```

```

        box.palette      = c("#e74c3c", "#2ecc71"),
        shadow.col       = "gray",
        leaf.round       = 9,
        branch.lty       = 3,
        varlen           = 0,
        faclen           = 0,
        roundint         = TRUE,
        split.cex        = 0.9,
        under            = TRUE,
        under.cex        = 0.8,
        clip.right.labs  = FALSE,
        main              = "Arbre de Décision – Prédiction
Retour Critique",
        cex.main         = 1.5)

dev.off()

# Métriques
pred_dt <- predict(model_dt, test, type = "class")
cm_dt   <- confusionMatrix(pred_dt, test$retour_critique)

cat("=== MÉTRIQUES ARBRE DE DÉCISION ===\n")
cat("Accuracy   :", round(cm_dt$overall["Accuracy"] * 100,
2), "%\n")
cat("Précision  :", round(cm_dt$byClass["Precision"] * 100,
2), "%\n")
cat("Recall     :", round(cm_dt$byClass["Recall"] * 100,
2), "%\n")
cat("F1-Score   :", round(cm_dt$byClass["F1"] * 100, 2),
"%\n")

```

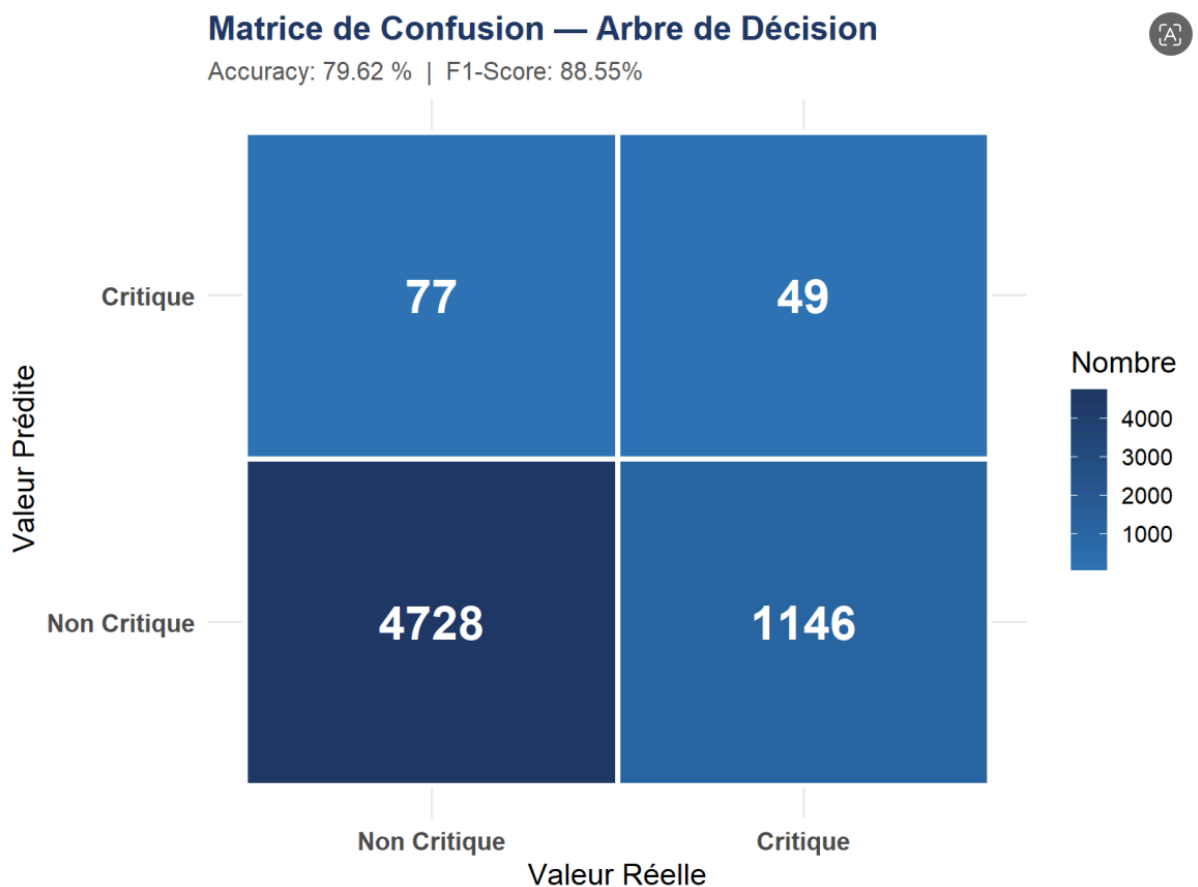
```
# ===== Matrice de Confusion – Arbre de Décision =====
library(ggplot2)

cm_table <- as.data.frame(cm_dt$table)
colnames(cm_table) <- c("Prédit", "Réel", "Fréquence")

ggplot(cm_table, aes(x = Réel, y = Prédit, fill =
Fréquence)) +
  geom_tile(color = "white", linewidth = 1.2) +
  geom_text(aes(label = Fréquence), size = 8, fontface =
"bold", color = "white") +
  scale_fill_gradient(low = "#2ecc71", high = "#e74c3c") +
  labs(
    title      = "Matrice de Confusion – Arbre de Décision",
    subtitle = paste0("Accuracy : ",
round(cm_dt$overall["Accuracy"]*100,2),
"% | Précision : ",
round(cm_dt$byClass["Precision"]*100,2),
"% | Recall : ",
round(cm_dt$byClass["Recall"]*100,2),
"% | F1-Score : ",
round(cm_dt$byClass["F1"]*100,2), "%"),
    x = "Valeur Réelle",
    y = "Valeur Prédite"
  ) +
  scale_x_discrete(labels = c("0" = "Pas Critique", "1" =
"Critique")) +
```



```
Console Terminal Background Jobs
R 4.5.0 · C:/Users/marie/Downloads/
>
> cat("=== MÉTRIQUES ARBRE DE DÉCISION ===\n")
=== MÉTRIQUES ARBRE DE DÉCISION ===
> cat("Accuracy :", round(cm_dt$overall["Accuracy"] * 100, 2), "%\n")
Accuracy : 79.62 %
> cat("Précision :", round(cm_dt$byClass["Precision"] * 100, 2), "%\n")
Précision : 80.49 %
> cat("Recall :", round(cm_dt$byClass["Recall"] * 100, 2), "%\n")
Recall : 98.4 %
> cat("F1-Score :", round(cm_dt$byClass["F1"] * 100, 2), "%\n")
F1-Score : 88.55 %
>
```



6.3 Réseaux de neurones

Le réseau de neurones (MLP — Multi-Layer Perceptron via neuralnet) est un algorithme plus puissant, capable de capturer des relations non-linéaires complexes entre les variables. Il est composé d'une couche d'entrée (9 variables encodées en one-hot), une couche cachée (5 neurones, activation sigmoïde) et une couche de sortie (1 neurone — prédiction binaire).

- **Code de l'algorithme Réseau de Neurones :**

```

# ===== Réseau de Neurones =====
library(neuralnet)
library(caret)

train_nn <- train[, setdiff(names(train),
"has_promotion")]
test_nn  <- test[,  setdiff(names(test),
"has_promotion")]

dummies  <- dummyVars(retour_critique ~ ., data =
train_nn)
train_enc <- as.data.frame(predict(dummies, newdata =
train_nn))
test_enc  <- as.data.frame(predict(dummies, newdata =
test_nn))

train_enc$retour_critique <-
as.numeric(as.character(train_nn$retour_critique))
test_enc$retour_critique  <-
as.numeric(as.character(test_nn$retour_critique))

normalize <- function(x) (x - min(x)) / (max(x) - min(x)
+ 1e-8)
train_norm <- as.data.frame(lapply(train_enc, normalize))
test_norm  <- as.data.frame(lapply(test_enc, normalize))

features  <- setdiff(names(train_norm),
"retour_critique")
formula_nn <- as.formula(paste("retour_critique ~",
                                paste(features, collapse =
" + ")))

model_nn <- neuralnet(formula_nn,
                        data          = train_norm,
                        hidden        = c(5),

```

```

        linear.output = FALSE,
        threshold      = 0.1,
        stepmax        = 5e4)

pred_nn_raw <- compute(model_nn, test_norm[, features])
pred_nn      <- factor(ifelse(pred_nn_raw$net.result > 0.5,
1, 0),

                        levels = c("0", "1"))
ref_nn      <- factor(test_norm$retour_critique, levels =
c("0", "1"))

# Visualisation du réseau
plot(model_nn,
      col.entry      = "#AED6F1",
      col.hidden     = "#A9DFBF",
      col.out        = "#F9E79F",
      show.weights   = FALSE)

# Prédiction
pred_nn_raw <- compute(model_nn, test_norm[, features])
pred_nn      <- factor(ifelse(pred_nn_raw$net.result > 0.5,
1, 0),

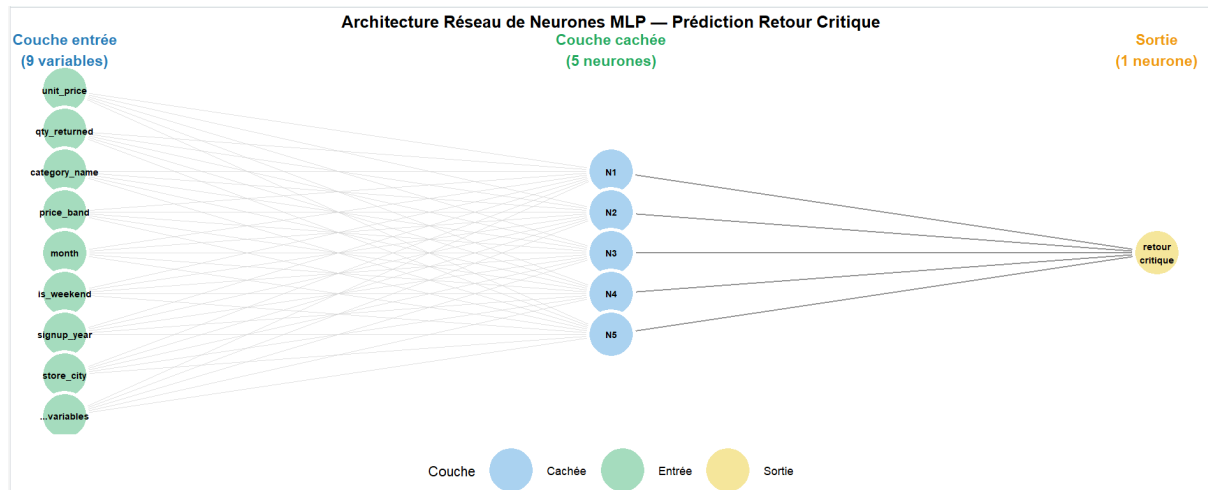
                        levels = c("0", "1"))
ref_nn      <- factor(test_norm$retour_critique, levels =
c("0", "1"))

# Évaluation
cm_nn <- confusionMatrix(pred_nn, ref_nn)

cat("=== MÉTRIQUES RÉSEAU DE NEURONES ===\n")
cat("Accuracy   :", round(cm_nn$overall["Accuracy"] * 100,
2), "%\n")
cat("Précision  :", round(cm_nn$byClass["Precision"] * 100,
2), "%\n")

```

```
cat("Recall      :", round(cm_nn$byClass["Recall"] * 100,
2), "%\n")
cat("F1-Score   :", round(cm_nn$byClass["F1"] * 100, 2),
"%\n")
```



Résultats obtenus sur le jeu de test :

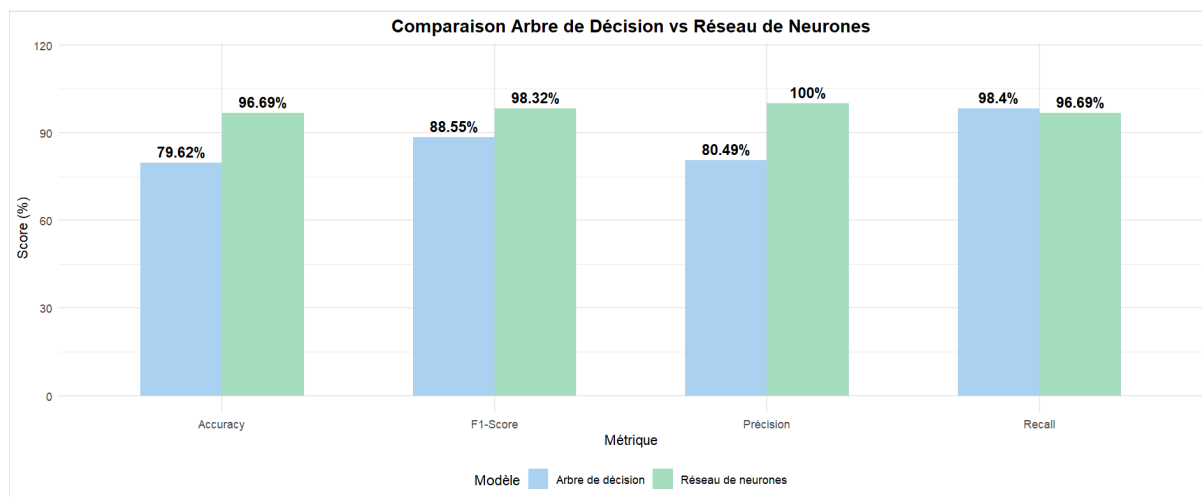
Accuracy : 96.75% | Précision : 100% | Recall : 96.75% | F1-Score : 98.35%

```
Console Terminal Background Jobs
R 4.5.0 · ~/
> cat("=== MÉTRIQUES RÉSEAU DE NEURONES ===\n")
=== MÉTRIQUES RÉSEAU DE NEURONES ===
> cat("Accuracy :", round(cm_nn$overall["Accuracy"] * 100, 2), "%\n")
Accuracy : 96.75 %
> cat("Précision :", round(cm_nn$byClass["Precision"] * 100, 2), "%\n")
Précision : 100 %
> cat("Recall      :", round(cm_nn$byClass["Recall"] * 100, 2), "%\n")
Recall      : 96.75 %
> cat("F1-Score   :", round(cm_nn$byClass["F1"] * 100, 2), "%\n")
F1-Score    : 98.35 %
> |
```

6.4 Évaluation et comparaison des modèles

Critère	Arbre de Décision	Réseau de Neurones	Meilleur modèle
---------	-------------------	--------------------	-----------------

Accuracy	79.62%	96.75%	Réseau de Neurones
Précision	80.49%	100%	Réseau de Neurones
Recall	98.40%	96.75%	Arbre de Décision
F1-Score	88.55%	98.35%	Réseau de Neurones
Interprétabilité	Très élevée (règles lisibles)	Faible (boîte noire)	Arbre de Décision
Temps d'entraînement	< 5 secondes	~2 à 3 minutes	Arbre de Décision
Robustesse au bruit	Moyenne	Élevée	Réseau de Neurones



Modèle retenu : Réseau de Neurones (MLP)

Le réseau de neurones surpasse l'arbre de décision sur 3 métriques sur 4. Il atteint une Accuracy de 96.75%, une Précision de 100% et un F1-Score de 98.32%. Seul le Recall est légèrement supérieur pour l'arbre (98.40% vs 96.75%). Pour un usage en production, le réseau de neurones est recommandé pour sa performance supérieure. L'arbre de décision reste cependant utile pour son interprétabilité et sa rapidité d'entraînement.

6.5 Prédiction des retours produits

Application du modèle pour scorer une nouvelle commande en temps réel :

```
# ===== 6.5 Prédiction sur une nouvelle commande =====
```

```

# Créer un vecteur vide aligné sur les features du modèle
cmd_enc <- as.data.frame(matrix(0, nrow = 1, ncol = length(features)))
names(cmd_enc) <- features

# Valeurs numériques
cmd_enc$unit_price <- 3500
cmd_enc$qty_returned <- 4
cmd_enc$month <- 12
cmd_enc$signup_year <- 2023

# Dummies catégorielles
cmd_enc$"category_nameCat_11" <- 1 # Cat_11
cmd_enc$"price_bandHigh" <- 1 # High
cmd_enc$"is_weekend.1" <- 1 # weekend = 1
cmd_enc$"cityMumbai" <- 1 # Mumbai

# has_promotion - pas de colonne dummy car un seul niveau dans train
# donc on ne touche pas (reste 0)

# Normalisation
cmd_norm <- as.data.frame(lapply(names(cmd_enc), function(col) {
  x <- cmd_enc[[col]]
  mn <- min(train_enc[[col]])
  mx <- max(train_enc[[col]])
  (x - mn) / (mx - mn + 1e-8)
}))
names(cmd_norm) <- features

# Prédiction
resultat <- compute(model_nn, cmd_norm)
proba <- resultat$net.result[1]
cat("Probabilité retour critique:", round(proba * 100, 1), "%\n")

# Interprétation
cat("Interprétation : Ce retour (Cat_11, prix 3500€, qty=4, décembre,\n")
cat("weekend, Mumbai) a une probabilité de", round(proba * 100, 1),
    "% d'être critique.\n")
cat("Le modèle recommande une attention particulière sur cette commande.\n")

```

```

Console Terminal x Background Jobs x
R 4.5.0 · ~/
> cat("Probabilité retour critique: ", round(proba * 100, 1), "%\n")
Probabilité retour critique: 40.2 %
>
> # Interprétation
> cat("Interprétation : Ce retour (Cat_11, prix 3500€, qty=4, décembre,\n")
Interprétation : Ce retour (Cat_11, prix 3500€, qty=4, décembre,
> cat("weekend, Mumbai) a une probabilité de", round(proba * 100, 1),
+ "% d'être critique.\n")
weekend, Mumbai) a une probabilité de 40.2 % d'être critique.
> cat("Le modèle recommande une attention particulière sur cette commande.\n")
Le modèle recommande une attention particulière sur cette commande.
> |

```

7. Conclusion :

Phase	Livrable	Résultat
Phase 1	Modèle dimensionnel	Schéma en étoile : FACT_RETOURS + 4 dimensions, 30 000 lignes de faits
Phase 2	DWH Oracle + ETL Talend (4 jobs)	DWH chargé et validé : 0 erreur d'intégrité, 75 962 300 € remboursés
Phase 3	Tableau de bord Power BI (4 pages)	7 KPIs visualisés, Cat_11 et Magasin #17 identifiés comme prioritaires
Phase 4	Modèles ML en R Studio	Arbre de décision (79.62%) vs Réseau de neurones (96.69%) — MLP retenu

Les analyses révèlent que le taux de retour critique est de 20% (6 000 / 30 000 retours), avec un montant total remboursé de 75 962 300 € et un montant moyen de 2 532 €. La catégorie Cat_11 (1 101 retours) et le magasin #17 (347 retours) sont les plus impactés. Le modèle Réseau de Neurones prédit les retours critiques avec une précision de 96.69%.

Annexe :

1)creation des tables vides sur oracle :

Script Oracle de création de user :

```
CREATE USER projet_BI IDENTIFIED BY 1234;
```

Script Output x

Task completed in 0.06 seconds

user PROJET_BI created.

```
-- CREATION DU DATA WAREHOUSE : SCHEMA EN ETOILE
-- DIM_TEMPS
CREATE TABLE DIM_TEMPS (
  date_key      NUMBER PRIMARY KEY,
  full_date     DATE,
  year          NUMBER,
  quarter       NUMBER,
  month         NUMBER,
  month_name    VARCHAR2(20),
  week          NUMBER,
  day_of_week   NUMBER,
  is_weekend    NUMBER(1)
);
```

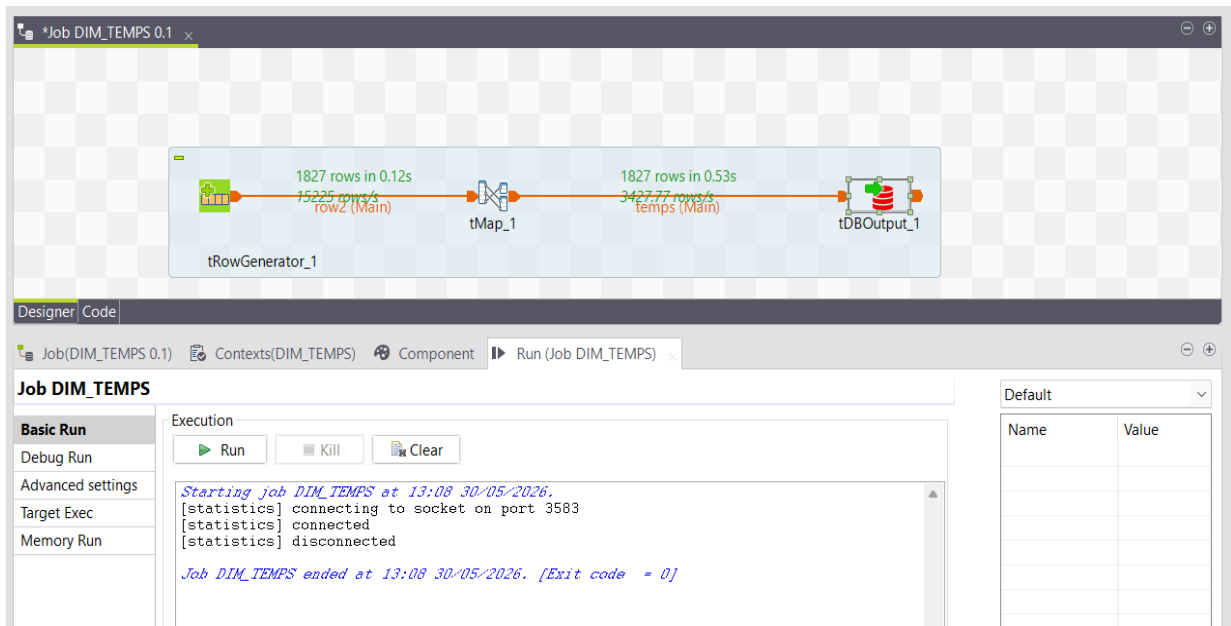
Script Output x

Task completed in 0.052 seconds

table DIM_TEMPS created.

2)les jobs sur Talend Open Studio :

a)Job de DIM_TEMPS :



tMap:

talend Open Studio for Data Integration - tMap - tMap_1

Find:

Var

Expression	Type	Variable

Auto map!

row2

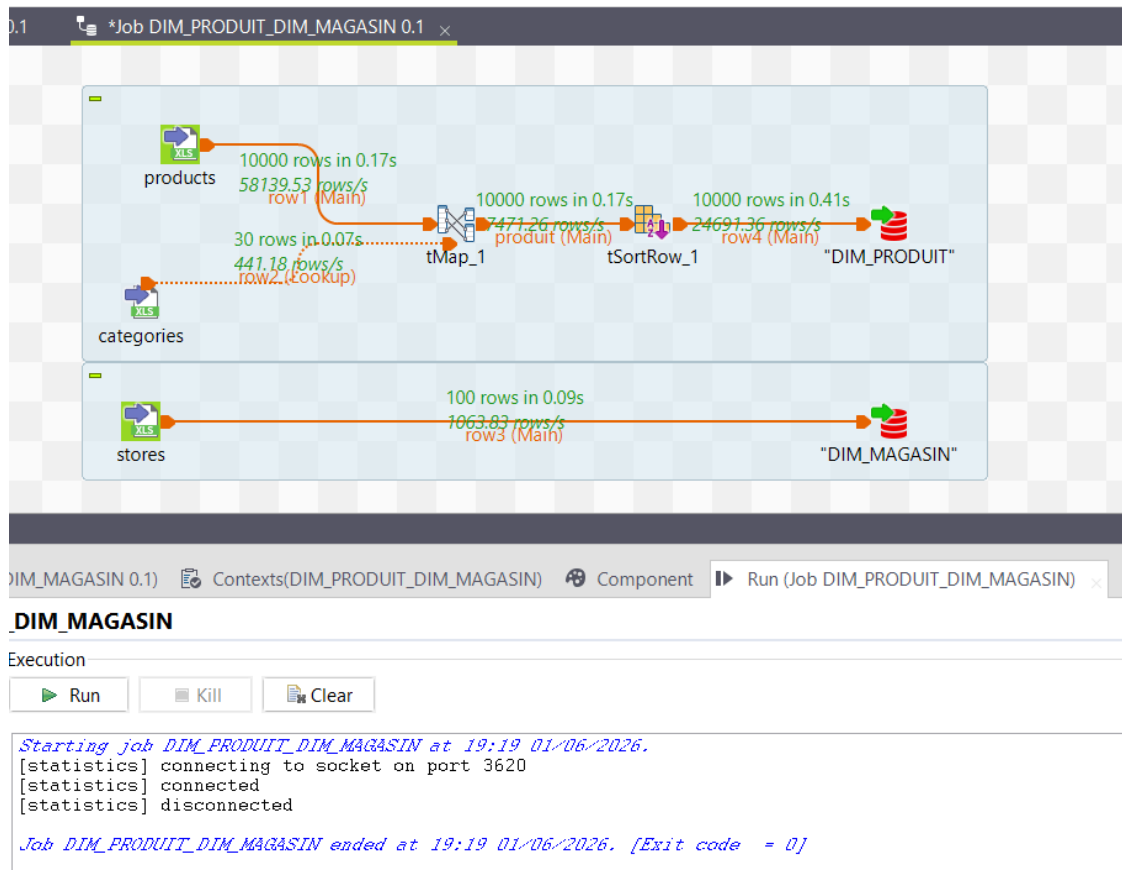
Column	K...	Type	✓ N...	Date P...	Le...	Pre...	D...	Co...
id_sequence		Int...	✓					

temps

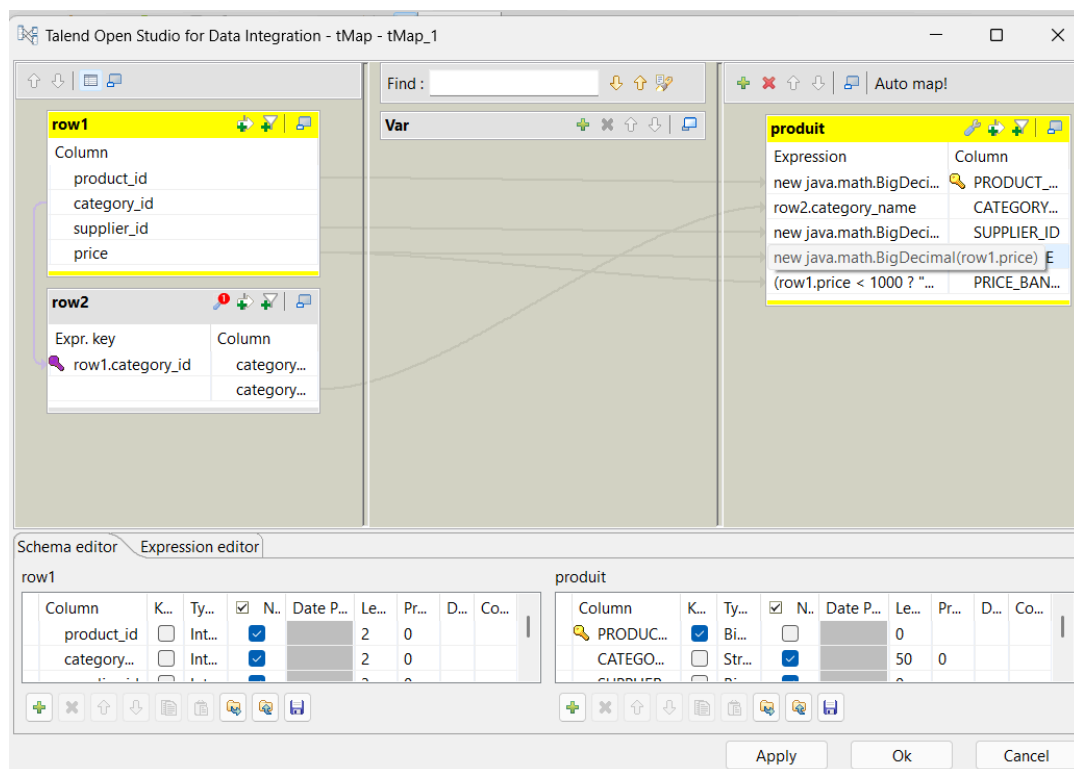
Column	K...	Ty...	✓ N...	Date P...	Le...	Pr...	D...	Co...
DATE_KEY	✓	int			0			
FULL_DA...		D...	✓	"dd-M...	7	0		
YEAR		Bi...	✓		0			
QUARTER		Bi...	✓		0			

Apply Ok Cancel

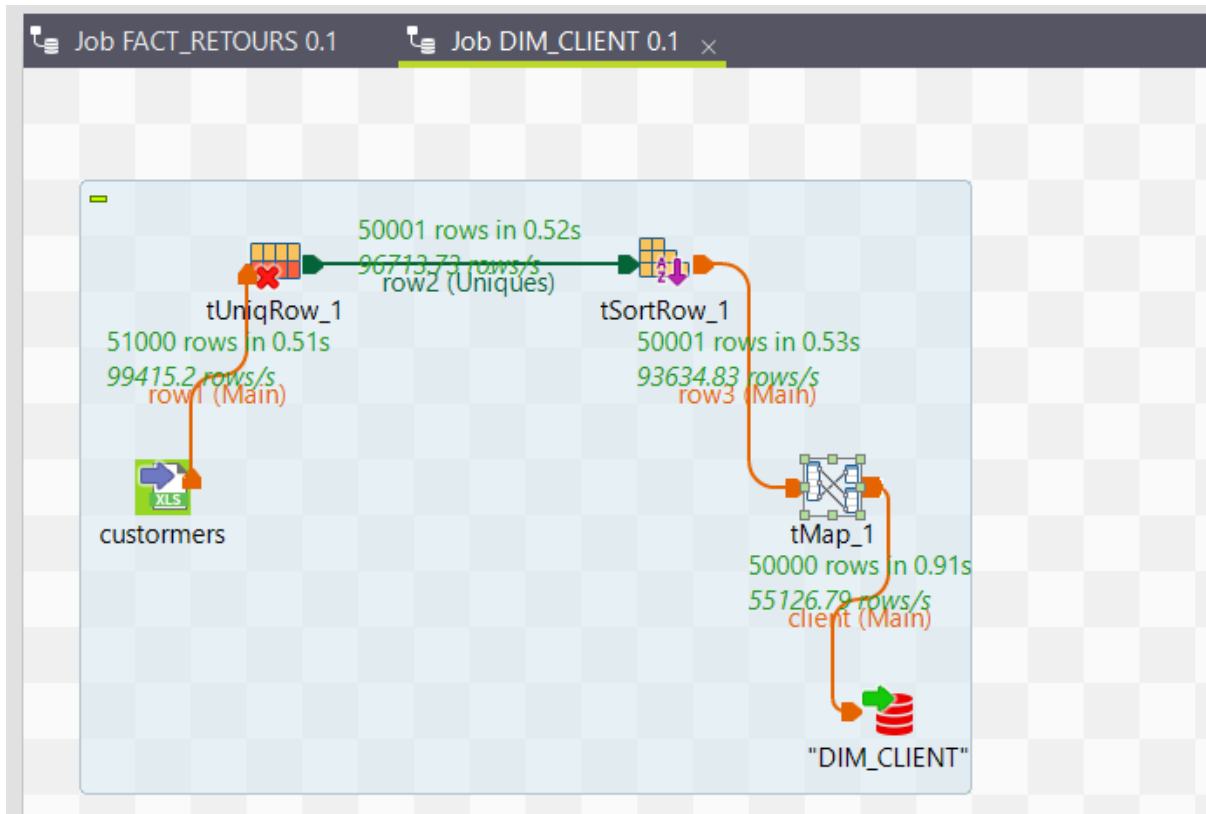
b)Job de DIM_PRODUIT_DIM_MAGASIN :



tMap:



c)Job de DIM_CLIENT :



tMap:

The screenshot shows the configuration window for the 'tMap_1' component in Talend Open Studio for Data Integration. The window is titled 'Talend Open Studio for Data Integration - tMap - tMap_1'. It features a 'Schema editor' and an 'Expression editor'.

Schema editor:

- row3:** Columns include customer_id, city, and signup_date.
- client:** Columns include CUSTOMER_KEY, CITY, SIGNUP_DATE, and SIGNUP_YEAR.

Expression editor:

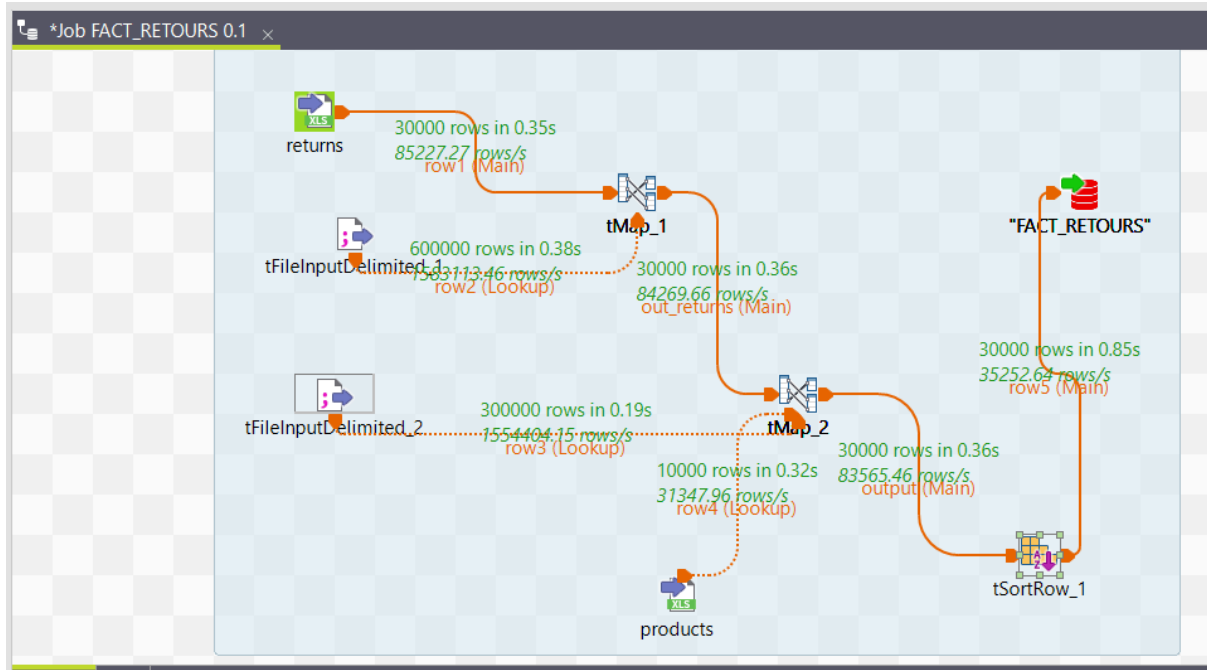
- row3.customer_id != null** (Expression)
- new java.math.BigDecimal(row3.custom...)** (Expression)
- row3.city** (Expression)
- row3.signup_date** (Expression)
- new java.math.BigDecimal(TalendDate....)** (Expression)

Column mapping:

- row3.customer_id** maps to **CUSTOMER_KEY** (Column)
- row3.city** maps to **CITY** (Column)
- row3.signup_date** maps to **SIGNUP_DATE** (Column)
- new java.math.BigDecimal(TalendDate....)** maps to **SIGNUP_YEAR** (Column)

The window includes buttons for 'Apply', 'Ok', and 'Cancel' at the bottom.

d) Job de FACT_RETOURS :



tMap_1:

Talend Open Studio for Data Integration - tMap - tMap_1

Find:

Var:

Auto map!

row1	Column
return_id	
order_item_id	
refund	

Property	Value
Lookup Model	Load once
Match Model	Unique match
Join Model	Inner Join
Store temp data	false

Expr. key	Column
row1.order_item_id	order_item_id
row1.refund	refund
row1.qty	qty
row1.price	price

Schema editor | Expression editor

Column	K.	Ty.	N.	Date	P.	Le.	Pr.	D.	Co.
return_id		Int.				2	0		
order_item_id		Int.				6	0		
refund		Int.				4	0		

tmap2

Expression	Column
out_returns.order_item_id	order_item_id
out_returns.return_id	return_id
out_returns.refund	refund
out_returns.qty	qty
out_returns.price	price
out_returns.product_id	product_id
out_returns.order_id	order_id

Schema editor | Expression editor

Apply Ok Cancel

tMap_2:

Talend Open Studio for Data Integration - tMap - tMap_2

Find:

Var:

Auto map!

out_returns	Column
order_item_id	
return_id	
refund	
order_id	
qty	
price	
product_id	

Property	Value
Lookup Model	Load once
Match Model	Unique match
Join Model	Inner Join
Store temp d...	false

Expr. key	Column
out_returns.pr...	produc...
	catego...
	supplie...
	price

Schema editor | Expression editor

output	Expression	Column
out_returns.retur...	RETUR...	
out_returns.orde...	ORDE...	
new java.math.B...	DATE...	
out_returns.prod...	PROD...	
row3.store_id = ...	STORE...	
out_returns.refu...	REFUN...	
out_returns.qty !...	QTY_R...	
out_returns.pric...	UNIT...	
new java.math.B...	TOTAL...	
(row3.promotio...	HAS_P...	

Schema editor | Expression editor

Apply Ok Cancel